SOFTWARE DEVELOPMENT PROJECT (SDP)

Session 5 – UML Use Case Modeling

Use Case Diagram and Narratives

ExamEnv: actors, use cases and detailed scenarios

Project	ExamEnv – Infrastructure-as-Code for Exams
Document version	1.0
Date	29 October 2026
Group No.	Group 1
Team members	Gaurav Shukla – CloudOps and Agents Gaurav Singh – Backend and Research Manya Tiwari – Backend and Research Nandani – ML (Parser)
Course	Software Engineering / SDP – CA-2
Status	Draft for submission

1. Introduction

This document models the functional behaviour of ExamEnv from the user's point of view using UML use cases. It contains the actors, the use case list, the use case diagram and a detailed narrative for each use case. Requirement numbers (FR-xx) refer to the SRS from Session 4.

1.1 Actors

Actor	Type	Description
Teacher	Primary	Creates exams, uploads requirement PDFs, reviews and publishes specs, monitors the class.
Student	Primary	Pulls the published spec and provisions a verified environment using the Student Agent.
Admin	Primary	Manages user accounts and roles.
ML Parser (LLM service)	Secondary	Automated service that turns PDF text into YAML.
Docker Engine	Secondary	Runtime that builds images and runs containers on the student machine.

1.2 Use Case Identification

Use cases were identified by walking through the three phases of the project data flow (teacher upload and parsing, student provisioning, teacher monitoring) and asking, for each actor, which goal they want to achieve with the system. Supporting steps that are always part of a bigger goal were modelled with include; optional behaviour was modelled with extend.

ID	Use case	Primary actor	Relationship
UC01	Register / Login	Teacher, Student, Admin	-
UC02	Upload Requirement PDF	Teacher	-
UC03	Parse PDF to YAML Spec	ML Parser (LLM service)	included by UC02
UC04	Review and Edit YAML Spec	Teacher	-
UC05	Publish Environment Spec	Teacher	-
UC06	Monitor Class Readiness Status	Teacher	-
UC07	View Student Error Logs	Teacher (extends UC06)	extends UC06
UC08	View and Pull Exam Spec	Student	-
UC09	Dry-Run Preview and Confirm	Student (included by UC10)	included by UC10
UC10	Provision Environment	Student, Docker Engine	-
UC11	Verify Environment	Docker Engine	included by UC10
UC12	Report Status to Backend	Student Agent (system step)	included by UC10

ID	Use case	Primary actor	Relationship
UC13	Manage Users and Roles	Admin	-
UC14	Use Template Library (future)	Teacher	extends UC02

2. Use Case Diagram

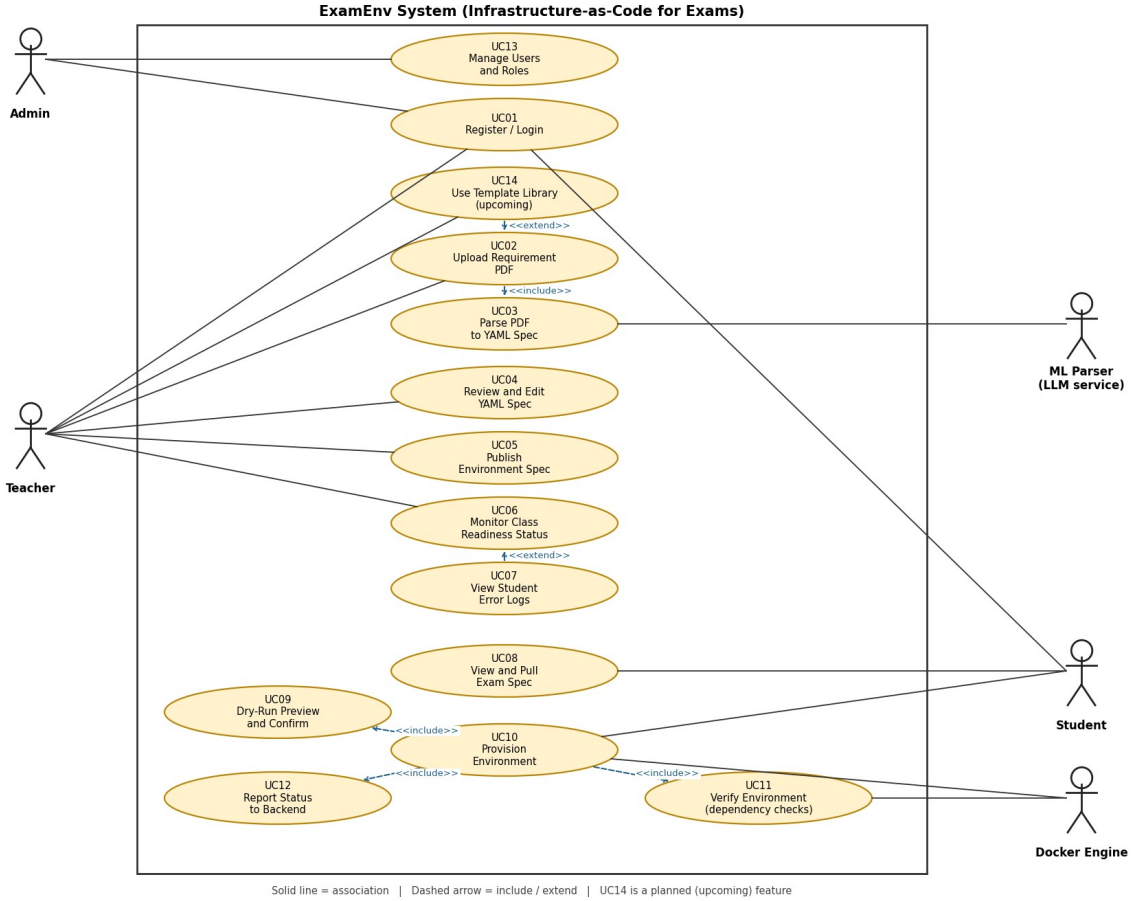


Figure 1: ExamEnv use case diagram. The editable version is use_case_diagram.drawio (open in draw.io or import into StarUML).

2.1 Reading the diagram

- Solid lines connect actors to the use cases they take part in.
- <<include>>: UC02 always uses UC03; UC10 always uses UC09, UC11 and UC12.
- <<extend>>: UC07 optionally extends UC06 (only when a student has failed); UC14 optionally extends UC02.
- Secondary actors (ML Parser, Docker Engine) are on the right because the system calls them, not the other way round.

3. Use Case Narratives

UC01: Register / Login

Actor(s)	Teacher, Student, Admin
Description	Allows a user to create an account and log in so that role-based features become available.
Priority	Must
Preconditions	Backend is running. For login the account already exists and is active.
Trigger	User opens the dashboard login page or runs the agent login command.
Main flow	1. User enters name, email and password (registration) or email and password (login). 2. System validates the input format. 3. System checks credentials; passwords are compared as salted hashes. 4. System issues a time-limited access token and loads the page for the user's role.
Alternate flows	A1: New user chooses Register; system creates the account with role Student or Teacher after validation (Admin approval may be required for Teacher).
Exceptions	E1: Wrong password or unknown email: system shows an error and allows retry; repeated failures are logged. E2: Account deactivated: login refused with a message.
Postconditions	User is authenticated with a valid token; the action is logged.
Requirements	FR-01, FR-02, FR-03, FR-04, FR-21

UC02: Upload Requirement PDF

Actor(s)	Teacher
Description	Teacher creates an exam and uploads the plain-language requirement document that describes the software needed.
Priority	Must
Preconditions	Teacher is logged in.
Trigger	Teacher clicks Upload on the dashboard.
Main flow	1. Teacher creates or selects an exam (title, course code, time window). 2. Teacher selects a PDF file and submits. 3. System checks that the file is a PDF of at most 20 MB. 4. System stores the file (S3 or local disk) and saves its metadata. 5. System queues a parse job (includes UC03) and shows 'Parsing started'. 6. Teacher can continue using the dashboard while parsing runs.
Alternate flows	A1: Teacher chooses a template from the Template Library instead of uploading (UC14, future).

Exceptions	E1: Wrong file type or file too large: upload rejected with a clear message. E2: Storage unavailable: system reports the error and nothing is saved.
Postconditions	Document and exam are stored; a parse job is queued.
Requirements	FR-06, FR-07, FR-08, FR-09, FR-10

UC03: Parse PDF to YAML Spec

Actor(s)	ML Parser (LLM service); triggered by UC02
Description	Converts the uploaded PDF into a structured draft YAML specification.
Priority	Must
Preconditions	A parse job exists for an uploaded document.
Trigger	Celery worker picks up the queued job.
Main flow	1. Worker reads the PDF and extracts its text. 2. ML Parser identifies packages, versions, environment variables, ports, datasets and setup commands. 3. ML Parser builds YAML following the ExamEnv schema. 4. System validates the YAML against the schema. 5. System saves it as a Draft spec and marks the job Completed. 6. System notifies the Teacher that the draft is ready for review.
Alternate flows	A1: Parser output is incomplete; system saves what was found and flags missing fields for the Teacher.
Exceptions	E1: PDF has no readable text or the LLM service is down: job is marked Failed, retried once automatically, then the Teacher is told and may enter YAML manually. E2: YAML fails schema validation: errors are stored and shown in the editor.
Postconditions	A Draft spec linked to the exam exists, or the job is Failed with an error message.
Requirements	FR-09, FR-11 to FR-15

UC04: Review and Edit YAML Spec

Actor(s)	Teacher
Description	Teacher checks the AI-generated YAML, corrects mistakes and saves the draft (human-in-the-loop).
Priority	Must
Preconditions	A Draft spec exists for the exam.
Trigger	Teacher opens the draft from the dashboard.

Main flow	1. System shows the YAML in an editor with validation messages. 2. Teacher reads each package, port and command and edits any mistake. 3. System re-validates the YAML as the Teacher types. 4. Teacher saves the draft; system increments the version.
Alternate flows	A1: Teacher discards the draft and re-uploads a corrected PDF.
Exceptions	E1: Invalid YAML: Save is blocked until errors are fixed. E2: Session expired: Teacher logs in again; unsaved edits are offered back.
Postconditions	The updated Draft spec is saved with a new version.
Requirements	FR-15, FR-16, FR-17, FR-19

UC05: Publish Environment Spec

Actor(s)	Teacher
Description	Makes a reviewed spec visible to students.
Priority	Must
Preconditions	A valid Draft spec exists.
Trigger	Teacher clicks Publish.
Main flow	1. System shows a final summary of the spec and asks for confirmation. 2. Teacher confirms. 3. System changes the spec state to Published and records who published it and when. 4. Students can now see and pull this spec.
Alternate flows	A1: Teacher unpublishes or edits later; a new version is created and the older one is kept in history.
Exceptions	E1: Spec invalid: publish is refused with the list of errors.
Postconditions	Spec is Published and visible to students.
Requirements	FR-18, FR-19

UC06: Monitor Class Readiness Status

Actor(s)	Teacher
Description	Teacher watches the live status of every student.
Priority	Must
Preconditions	Spec is Published and the Teacher is logged in.
Trigger	Teacher opens the exam status page.

Main flow	1. System loads the enrolled students and their latest status. 2. Dashboard shows Ready (green), Failed (red) or Not Started (grey) for each student. 3. Dashboard opens a WebSocket connection. 4. When an agent reports, the backend pushes the update and the row changes within 5 seconds. 5. Summary counts are updated.
Alternate flows	A1: Teacher filters or searches by status (could-have).
Exceptions	E1: WebSocket drops: dashboard reconnects automatically and reloads the latest state.
Postconditions	Teacher sees an up-to-date view of class readiness.
Requirements	FR-33, FR-34, FR-36, FR-37

UC07: View Student Error Logs

Actor(s)	Teacher (extends UC06)
Description	Teacher opens the exact error log of a failed student to help them.
Priority	Must
Preconditions	A student's status is Failed.
Trigger	Teacher clicks the failed student's row.
Main flow	1. System retrieves the verification log and error message for that student. 2. Dashboard displays it (for example 'Port 5432 already in use'). 3. Teacher uses the message to guide the student.
Alternate flows	A1: Teacher downloads the log as text.
Exceptions	E1: No log stored: system shows 'No log available' and suggests re-running the agent.
Postconditions	Teacher knows the cause of the failure.
Requirements	FR-35

UC08: View and Pull Exam Spec

Actor(s)	Student
Description	Student sees available exams and downloads the published YAML.
Priority	Must
Preconditions	Student is logged in through the agent; Docker is installed.
Trigger	Student runs the pull command.
Main flow	1. Agent fetches the list of Published exams from the backend. 2. Student selects an exam. 3. Agent downloads the Published YAML and stores it locally.

Alternate flows	A1: Only one exam is available; agent selects it automatically.
Exceptions	E1: No Published spec yet: agent says so and asks the student to try later. E2: Network error: agent shows the problem and retries on request.
Postconditions	The spec is available on the student's machine.
Requirements	FR-22, FR-23

UC09: Dry-Run Preview and Confirm

Actor(s)	Student (included by UC10)
Description	Shows exactly what will be installed or changed before anything runs.
Priority	Must
Preconditions	Spec has been pulled.
Trigger	Student runs provision, which starts with a dry-run.
Main flow	1. Agent reads the YAML. 2. Agent prints a preview: base image, packages and versions, ports, volume mounts and setup commands. 3. Agent asks 'Proceed? (Yes/No)'. 4. Student answers Yes.
Alternate flows	A1: Student answers No; agent stops and changes nothing.
Exceptions	E1: Preview detects a port already in use: agent warns before continuing.
Postconditions	Student has approved the plan, or the process is stopped.
Requirements	FR-24

UC10: Provision Environment

Actor(s)	Student, Docker Engine
Description	Builds and starts the isolated Docker container described by the spec.
Priority	Must
Preconditions	Student confirmed the dry-run (UC09).
Trigger	Student confirms Yes.
Main flow	1. Agent builds a Docker image from the spec or uses a base image and installs packages. 2. Agent starts the container with the exposed ports and mounts. 3. Agent runs verification (UC11). 4. Agent reports the result (UC12).
Alternate flows	A1: Student chooses to retry or clean up an old environment first.
Exceptions	E1: Docker not running or build fails: agent shows the error, marks status Failed and reports it. E2: Port conflict or low disk space: agent stops with advice.

Postconditions	A running container exists and the result is verified and reported, or the status is Failed with logs.
Requirements	FR-25, FR-26, FR-27, FR-28

UC11: Verify Environment

Actor(s)	Docker Engine; run by the Student Agent
Description	Automatically checks that everything the spec asked for is present.
Priority	Must
Preconditions	Container is running.
Trigger	Provisioning finishes.
Main flow	1. Agent runs the verification script inside the container. 2. Script checks versions (for example python --version), pip list and imports. 3. Agent compares each result with the spec and marks pass or fail. 4. Agent collects the logs.
Alternate flows	A1: All checks pass; status becomes Ready.
Exceptions	E1: One or more checks fail; status becomes Failed and the failing checks are listed.
Postconditions	A pass/fail result and logs are available.
Requirements	FR-29, FR-30, FR-31

UC12: Report Status to Backend

Actor(s)	Student Agent (system step)
Description	Sends the final status and logs to the backend so the teacher can see them.
Priority	Must
Preconditions	Verification has finished.
Trigger	Verification completes.
Main flow	1. Agent sends status and logs through the REST API. 2. Backend stores them and updates the student's status record. 3. Backend broadcasts the change to the dashboard.
Alternate flows	A1: Network is down; agent keeps the result and resends when connected.
Exceptions	E1: Backend returns an error; agent retries a few times and then tells the student.
Postconditions	The Teacher dashboard shows the student's new status.
Requirements	FR-32

UC13: Manage Users and Roles

Actor(s)	Admin
Description	Admin creates, deactivates and assigns roles to user accounts.
Priority	Must
Preconditions	Admin is logged in.
Trigger	Admin opens the user management page.
Main flow	1. Admin views the user list. 2. Admin creates a user or selects an existing one. 3. Admin sets the role (Teacher, Student, Admin) or deactivates the account. 4. System saves the change and logs it.
Alternate flows	A1: Admin resets a user's password.
Exceptions	E1: Email already used: system rejects the duplicate.
Postconditions	User accounts and roles are updated.
Requirements	FR-03, FR-05

UC14: Use Template Library (future)

Actor(s)	Teacher
Description	Teacher starts from a pre-built verified YAML template for a common course type instead of an AI draft.
Priority	Could
Preconditions	Teacher is logged in; templates exist.
Trigger	Teacher selects Use Template while creating an exam.
Main flow	1. System lists templates by category (Python, Web Dev, Databases, ML). 2. Teacher selects one. 3. System copies it into a Draft spec for review (UC04).
Alternate flows	A1: Teacher edits the template before saving.
Exceptions	E1: Template not found: system shows the list again.
Postconditions	A Draft spec created from the template exists.
Requirements	FR-20